Deep Agents 实战

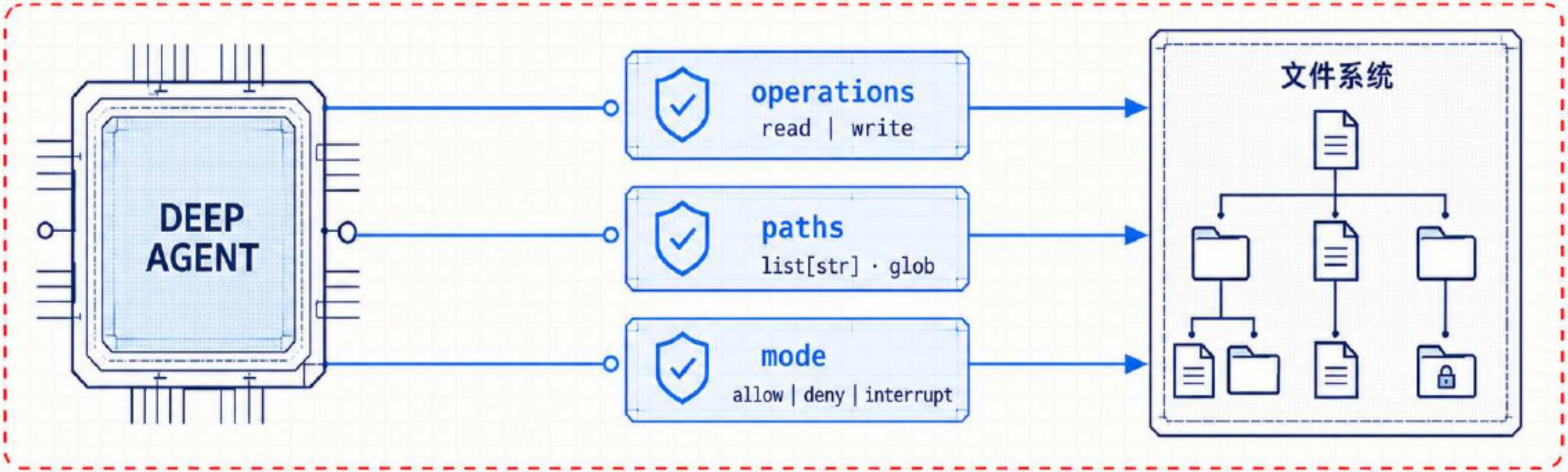
第 15 讲：文件系统权限

FS.PERM.015
SCHEMA v1.0
BLUEPRINT



文件能力就是副作用入口

权限配置必须回答三个问题



operations

read → ls · read_file · glob · grep

write → write_file · edit_file · delete

paths

list[str] · glob

mode

allow | deny | interrupt

在入口函数中启用文件权限

`create_deep_agent(..., permissions=[...])`

```python
from deepagents import FilesystemPermission, create_deep_agent

agent = create_deep_agent(
    model=model,
    backend=backend,
    permissions=[
        FilesystemPermission(
            operations=["write"],
            paths=["/**"],
            mode="deny",
        )
    ],
)
```

这条规则如何生效

operations=["write"]

仅匹配写操作：write_file / edit_file / delete

paths=["/"]**

匹配所有绝对路径

mode="deny"

命中规则后直接拒绝

结果：所有写操作被拒绝；读取因未匹配此规则而默认允许

规则顺序决定最终结果

first-match-wins；同时匹配操作与路径；无匹配时默认允许

请求

`write /workspace/app.py`

规则扫描轨道

Rule 1 `deny /workspace/secrets/**`	不匹配
Rule 2 `allow /workspace/**`	**匹配：允许，立即停止**
Rule 3 `deny /**`	不再检查

所有规则同时适用于 read + write。

伪白名单：

只有 `allow /workspace/**`，工作区外仍默认允许。

敏感路径拒绝 → 业务路径允许 → 全局拒绝

规则顺序交换会改变结果； | **最具体的规则放在前面。**

四种策略，明确操作与路径

以下为规则摘要，不是 Python 配置语法

全局只读

```
write  /**  →  deny
```

读取未命中，默认允许

工作区白名单

```
read + write /workspace/**
→ allow
```

```
read + write /** → deny
```

必须按此顺序

共享知识只读，用户记忆可写

```
write /policies/** → deny
```

```
/policies/** 可读
```

```
/memories/** 默认可读写
```

拒绝所有文件访问

```
read + write /** → deny
```

工具仍存在；
调用返回权限错误

allow / deny: deepagents >= 0.5.2； write 包含创建、整文件覆盖、编辑和删除。

interrupt 把敏感写入交给人

暂停 → 审查 → approve / edit / reject → 恢复



需要 deepagents ≥ 0.6.8

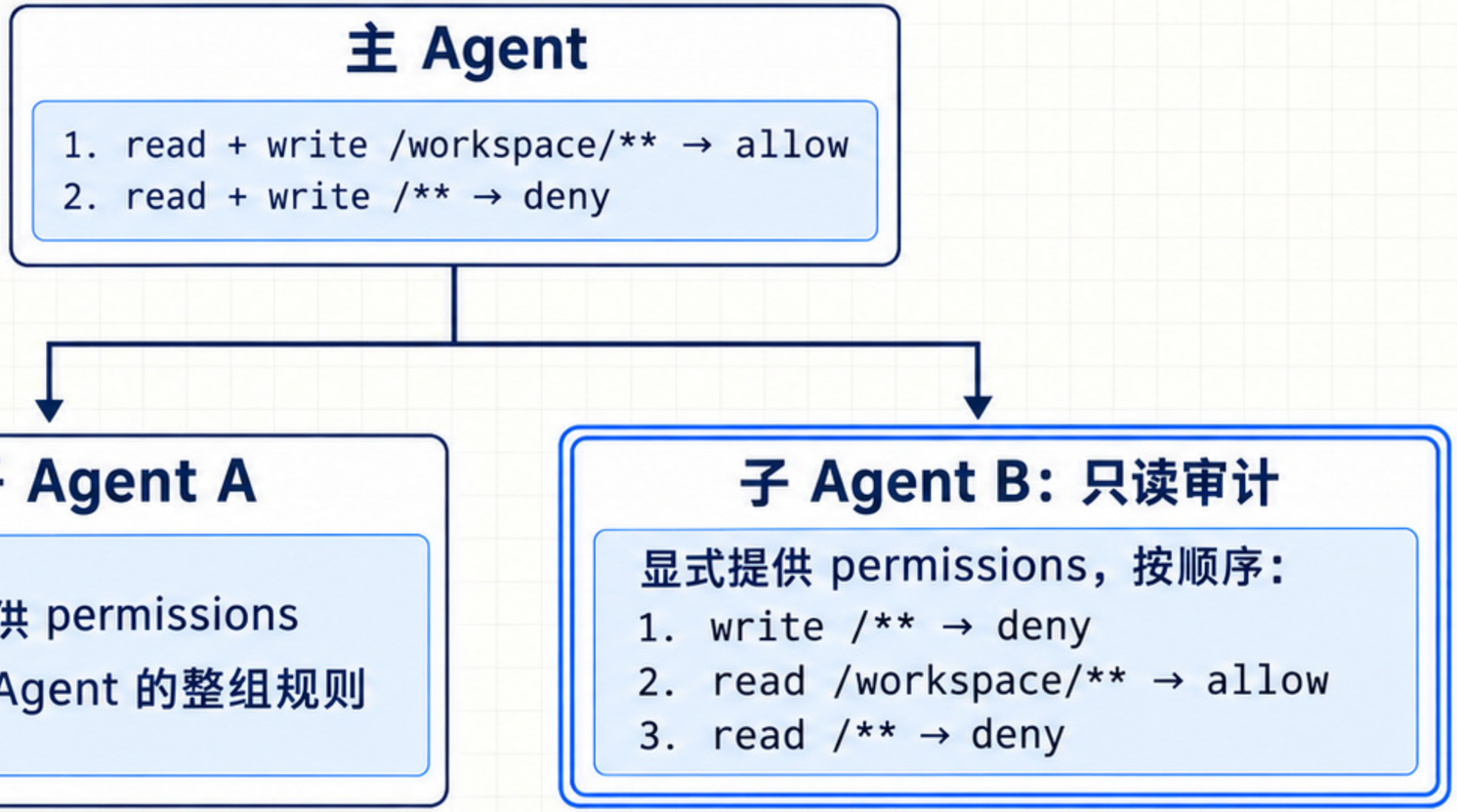
需要 checkpointer

恢复必须复用同一 thread_id

子 Agent 默认继承，显式配置整体替换

不是追加，也不是与父规则取交集

- 默认继承整组规则
- 显式配置整体替换
- 替换后必须独立闭合

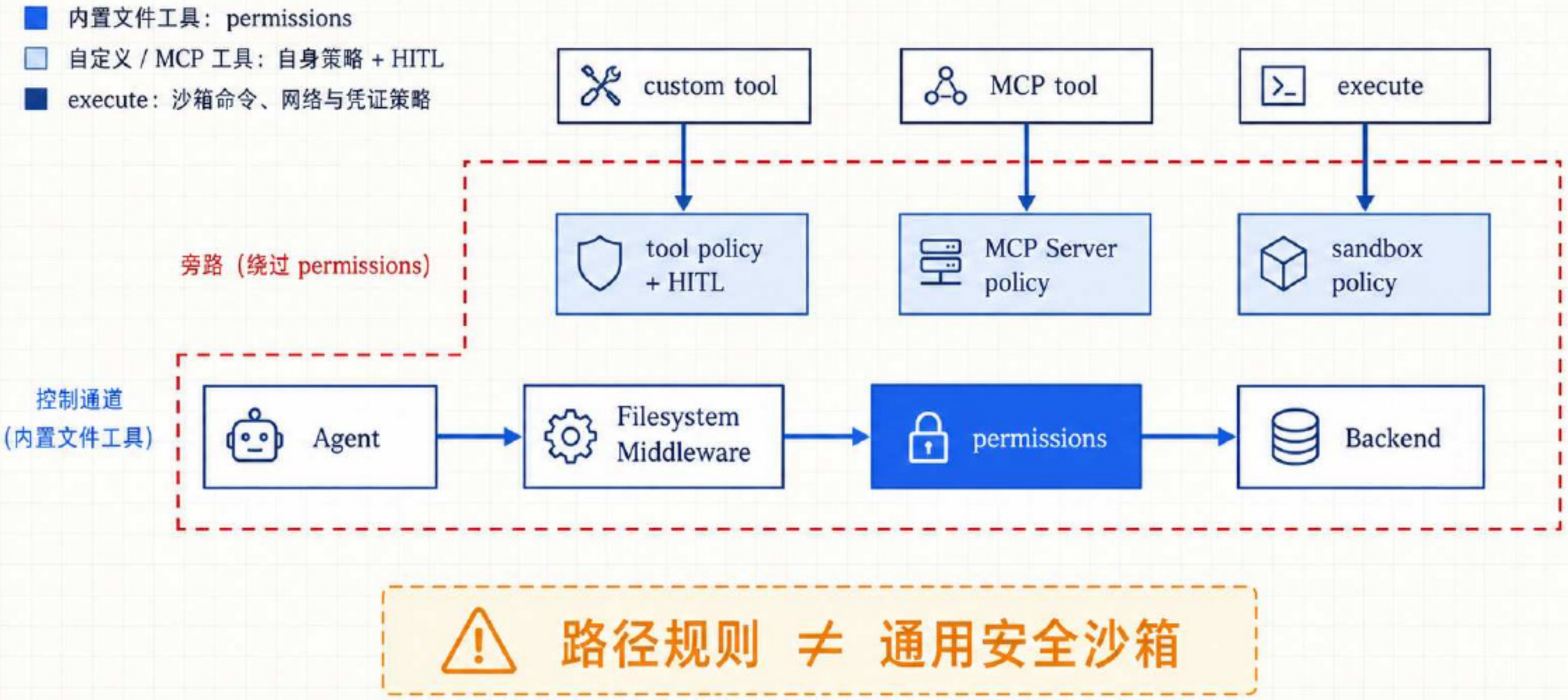


⚠ 只加“禁止写入”会丢失父规则的读取边界；替换后的规则必须独立闭合。

本图是规则摘要，不是实际配置语法。

权限只能守住内置文件工具

custom tool、MCP 与 execute 都要单独设防



⚠ 路径规则 ≠ 通用安全沙箱

上线前验证：检查实际文件，不只看回复

允许与拒绝都要测；版本升级后重新回归

允许路径：
正常读写不被误拦截

整文件覆盖：
已有文件仍受 write 规则保护

敏感路径：
读、写、编辑、删除均被拦截

目录删除：
包含受保护后代时整体失败

陌生路径：
全局拒绝真实生效

子 Agent / interrupt：
替换边界与同线程恢复正确

v0.7 同名 FilesystemMiddleware 会替换默认实例；不要假定顶层 permissions 自动合并。

grep / glob 要检查 truncated；自定义工具、MCP、execute 需要独立测试。

同时断言返回结果与文件内容，避免把模型回答当成权限验证。

CompositeBackend：沙箱默认路由的例外

文件工具权限不是 Shell 权限

架构

CompositeBackend 将请求分流：

- 默认 → 沙箱 [execute]
- /memories/ → 非 Shell 后端

不能覆盖沙箱默认路由

/workspace/** 或 /**

↓

构造时可能抛出 NotImplementedError

可保护已配置路由

write /memories/** → deny

↓

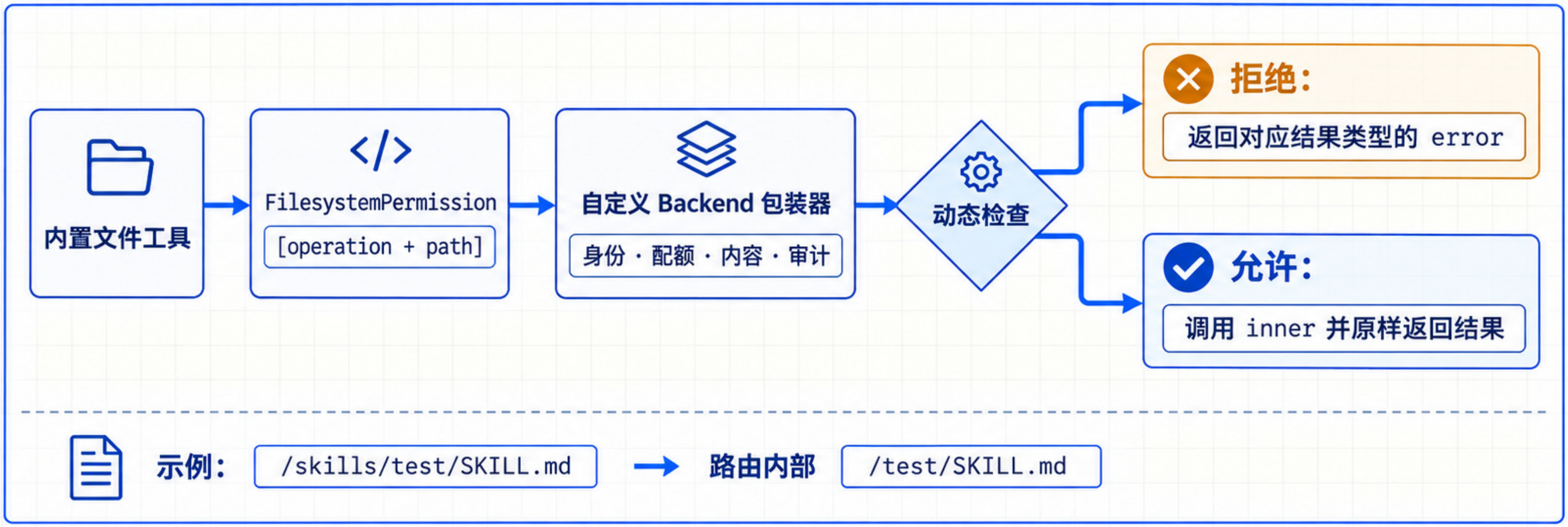
只约束内置文件工具

execute 不受 FilesystemPermission 控制；
用沙箱、命令、网络与凭证策略约束它。

核对 deepagents 0.7.22；
章节实验锁定 0.7.15。

Policy Hook：路径基线之外的动态策略

不是 create_deep_agent 的独立参数；PolicyWrapper 也不是内置可导入类



⚠ 完整覆盖写入、编辑、删除与所需协议方法

⚠ 路由内看到的是剥离前缀后的路径

⚠ 自定义工具、MCP 与 Shell 仍需另行控制

把最小权限写成可审查配置

稳定路径基线 · 少量人工审批 · 动态策略 · 沙箱隔离

- 先建立权限基线，再按 Agent 职责缩小边界

